

آزمایشگاه طراحی سیستم‌های دیجیتال

تابستان ۱۴۰۵
استاد: دکتر انصاری



دانشگاه صنعتی شریف
دانشکده مهندسی کامپیوتر

نام و کد دانشجویان: امیریوسف عبدی - ۴۰۳۱۰۶۲۸۴

آروین بقال اصل - ۴۰۳۱۰۵۷۹۳

پارسا عدل‌پرور - ۴۰۳۱۰۶۳۰۲

آزمایش اول

فهرست مطالب

۱	مقدمه آزمایش	۲
۲	طراحی مدار تشخیص بخش‌پذیری بر ۳ برای عدد ۴-رقمی BCD	۲
۱.۲	هدف مسئله	۲
۲.۲	تئوری مسئله	۲
۳.۲	پیاده‌سازی	۲
۱.۳.۲	ماژول XORR	۳
۲.۳.۲	ماژول BCDMod3	۳
۳.۳.۲	ماژول FA1	۴
۴.۳.۲	ماژول FA4	۵
۴.۲	ماژول نهایی Experiment1_DivBy3	۶
۳	طراحی مدار تشخیص بخش‌پذیری بر ۱۱ برای عدد ۴-رقمی BCD	۷
۱.۳	هدف مسئله	۷
۲.۳	تئوری مسئله	۷
۳.۳	پیاده‌سازی	۷
۱.۳.۳	ماژول FA6	۸
۲.۳.۳	ماژول DivBy11	۹
۴.۳	ماژول نهایی Experiment1_DivBy11	۱۰

۱ مقدمه آزمایش

شرکت‌های تولیدکننده تراشه‌های FPGA و CPLD، نرم‌افزارهایی را به عنوان ابزار طراحی به کمک کامپیوتر^۱ ارائه می‌دهند تا کاربران بتوانند از محصولات تولید شده توسط این شرکت‌ها استفاده کنند. هدف اصلی این آزمایش، آشنایی بیشتر با این ابزارهای CAD (به ویژه نرم‌افزار Quartus) و استفاده از امکانات شماتیک برای طراحی و پیاده‌سازی یک مدار ترکیبی است. این ابزارها علاوه بر پشتیبانی از زبان‌های استاندارد توصیف سخت‌افزار مانند Verilog و VHDL، امکان طراحی شماتیک را نیز فراهم می‌کنند. این قابلیت به طراحان اجازه می‌دهد تا مدارهای کوچک را با سرعت بالا و بدون نیاز به استفاده از زبان‌های توصیف سخت‌افزار، برای ابزار CAD تعریف و پیاده‌سازی کنند. در این آزمایش از این قابلیت برای بررسی و تشخیص مضارب اعداد استفاده خواهیم کرد. بدین منظور، دو مدار طراحی خواهیم کرد که مدار اول، بخش‌پذیری عدد BCD ورودی بر عدد ۳ را بررسی می‌کند، و مدار دوم بخش‌پذیری آن بر عدد ۱۱ را بررسی می‌کند.

۲ طراحی مدار تشخیص بخش‌پذیری بر ۳ برای عدد ۴-رقمی BCD

۱.۲ هدف مسئله

هدف از این بخش، طراحی یک مدار ترکیبی است که دارای یک خروجی یک‌بیتی باشد. عملکرد این مدار باید به این صورت باشد که خروجی آن تنها زمانی ۱ شود که ورودی BCD چهاررقمی اعمال‌شده، مضربی از عدد ۳ باشد. همچنین، محدودیت در نظر گرفته شده برای این طراحی این است که برای پیاده‌سازی مدار، فقط مجاز به استفاده از گیت‌های پایه هستیم.

۲.۲ تئوری مسئله

طبق قواعد ریاضی می‌دانیم عدد $N = \overline{x_1x_2x_3x_4}$ بر عدد ۳ بخش‌پذیر است، اگر و تنها اگر جمع ارقامش نیز، بر ۳ بخش‌پذیر است:

$$N \equiv 0 \pmod{3} \text{ where } N = \overline{x_1x_2x_3x_4} \iff x_1 + x_2 + x_3 + x_4 \equiv 0 \pmod{3}$$

۳.۲ پیاده‌سازی

از این ایده استفاده می‌کنیم که:

$$R = (x_1 + x_2 + x_3 + x_4) \pmod{3} =$$

$$((x_1 \pmod{3}) + (x_2 \pmod{3}) + (x_3 \pmod{3}) + (x_4 \pmod{3})) \pmod{3}$$

با استفاده از این قاعده ساده در ریاضی، می‌توانیم تعداد مازول‌هایی که نیاز به طراحی دارند را کمینه می‌کنیم. اولین مازولی که از آن استفاده می‌کنیم، مازول **BCDMod3** است. در ادامه طبق همان رابطه نوشته شده، نیاز به جمع‌کننده کامل^۲ داریم. برای اینکار کافیه یک مازول **جمع‌کننده کامل ۴-بیتی** بسازیم که برای ساخت خود این جمع‌کننده، نیاز به **جمع‌کننده کامل ۱-بیتی** داریم. این طراحی بهینه است چرا که در خاصیت استفاده‌پذیری مجدد^۳ مشاهده می‌شود و در محاسبه باقی‌مانده بر ۳ انتهای کار، می‌توانیم از همان **BCDMod3** استفاده کنیم. همچنین در مازول‌های **BCDMod3** و **جمع‌کننده کامل ۱-بیتی** از گیت XOR استفاده شده است که پیاده‌سازی آن در مازول **XORR** انجام شده است.

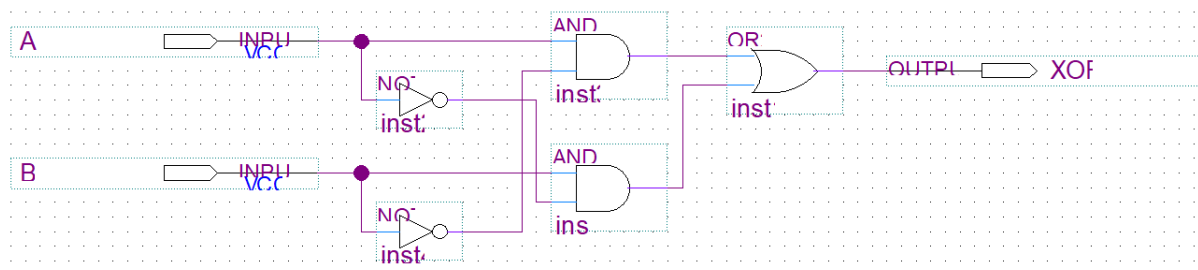
^۱CAD: Computer Aided Design

^۲FullAdder

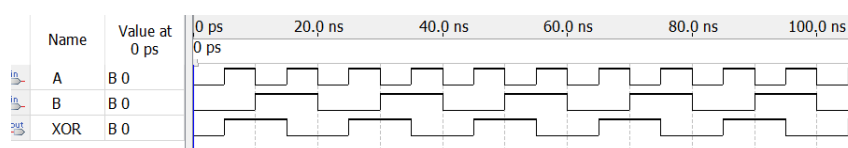
^۳Reusability

۱.۳.۲ ماژول XORR

- ورودی‌ها: A, B
- خروجی‌ها: XOR
- عملکرد: $XOR = (\bar{A} \cdot B) + (A \cdot \bar{B})$



شکل ۱: ماژول XORR



شکل ۲: عملکرد صحیح ماژول XORR

۲.۳.۲ ماژول BCDMod3

همانطور که در توضیحات بالا ذکر شد، از این ماژول برای محاسبه باقی مانده ارقام در ابتدای کار استفاده می‌کنیم و یکبار نیز در انتهای مراحل محاسبات، برای محاسبه باقی ماندن نهایی استفاده می‌کنیم. دلیل اینکه می‌توانیم در انتهای کار نیز از آن استفاده کنیم این است که مقدار $(x_1 \bmod 3) + (x_2 \bmod 3) + (x_3 \bmod 3) + (x_4 \bmod 3)$ در اصل حاصل جمع ۴ عدد است که این اعداد در بازه $[0, 2]$ قرار دارند. بنابراین حاصل نهایی در بازه $[0, 8]$ است که محدوده مجاز BCD می‌باشد. بنابراین می‌توانیم از این ماژول در انتهای کار استفاده کنیم.

- ورودی‌ها: A_1, A_2, A_3, A_4
- خروجی‌ها: $M_0, M_1, \text{DivBy3}$
- عملکرد: ابتدا برای سهولت محاسبات یک سیگنال میانی $X = A_1 \oplus A_3$ تعریف می‌کنیم. خروجی M_1 تنها در حالتی فعال است که باقی مانده ۲ باشد و این یعنی ورودی اعداد ۵، ۸ یا ۱۱ است. بنابراین:

$$M_1 = \bar{X} \cdot ((A_3 \oplus A_2) + A_4)$$

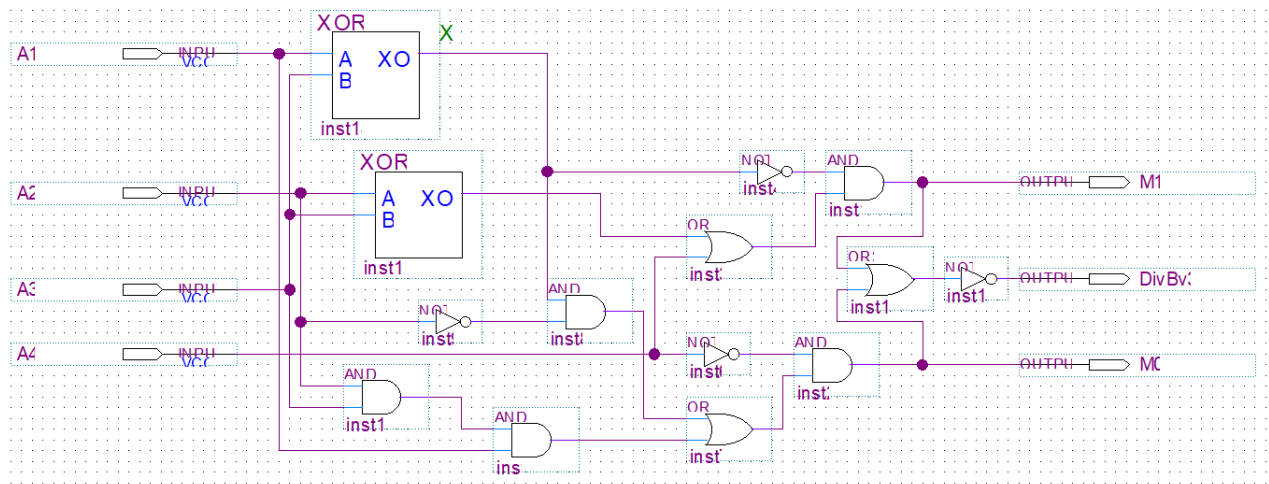
خروجی M_0 نیز برای اعداد ۱، ۴ و ۷ فعال می‌شود بنابراین:

$$M_0 = \bar{A}_4 \cdot ((\bar{A}_2 \cdot X) + (A_3 \cdot A_2 \cdot A_1))$$

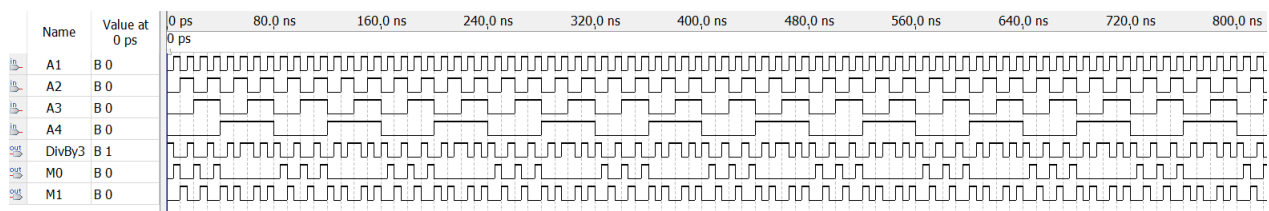
خروجی DivBy3 صرفاً در آخرین استفاده ما از این ماژول کاربرد دارد. وقتی که می‌خواهیم بررسی کنیم عدد ورودی بر ۳ بخش پذیر است یا خیر. دلیل تعریف این پین خروجی، این است که در بالاترین سطح^۴ نباید از گیت‌های منطقی استفاده کنیم.

$$\text{DivBy3} = \overline{M_0 + M_1}$$

Top-Level^۴



شکل ۳: ماژول BCDMod3



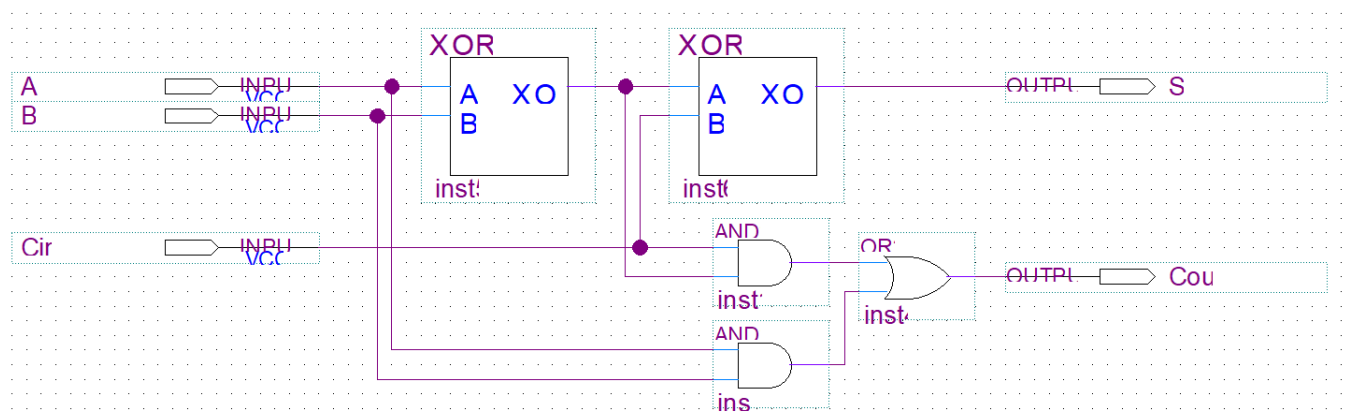
شکل ۴: عملکرد صحیح ماژول BCDMod3

۳.۳.۲ ماژول FA1

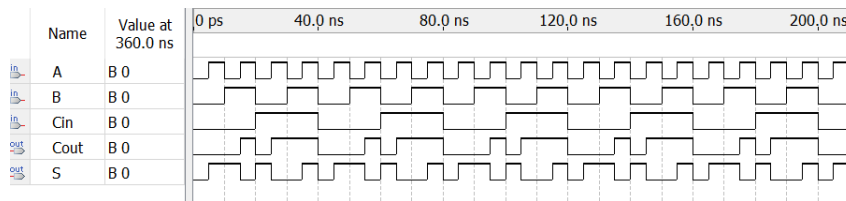
- ورودی‌ها: A, B, Cin
- خروجی‌ها: $S, Cout$
- عملکرد:

$$S = A \oplus B \oplus Cin$$

$$Cout = (A \cdot B) + (C \cdot (A \oplus B))$$



شکل ۵: ماژول FA1



شکل ۶: عملکرد صحیح ماژول FA1

۴.۳.۲ ماژول FA4

در این ماژول از ۴ عدد ماژول جمع‌کننده کامل ۱-بیتی استفاده کردیم و آن‌ها را بصورت جمع‌کننده آبشاری^۵ وصل کردیم.

• ورودی‌ها: $A_1, A_2, A_3, A_4, B_1, B_2, B_3, B_4, Cin$

• خروجی‌ها: $S_1, S_2, S_3, S_4, Cout$

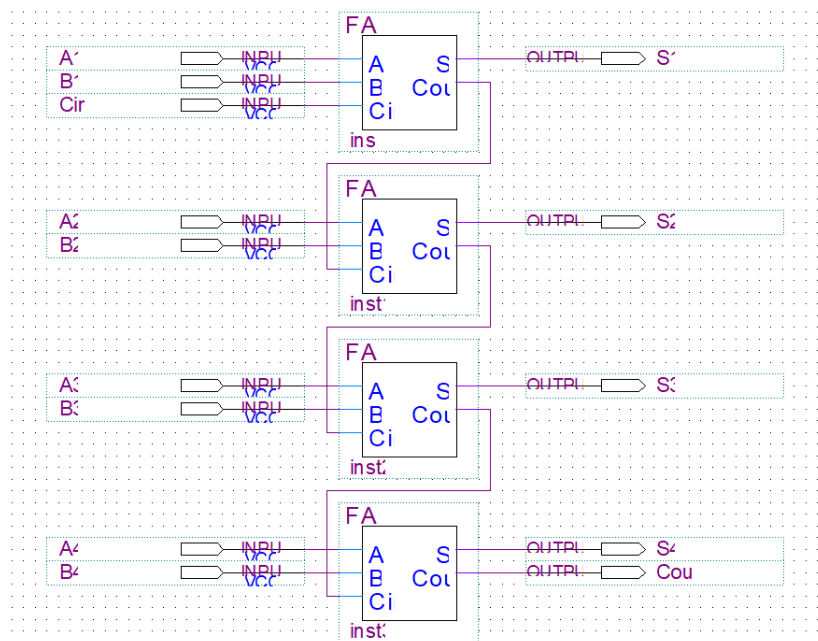
• عملکرد:

$$Cin_{FA1_0} = Cin$$

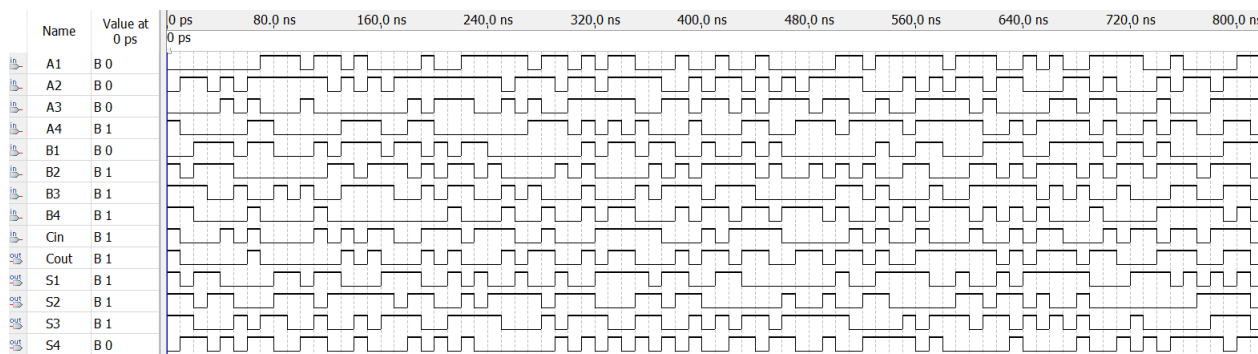
$$Cout = Cout_{FA1_3}$$

$$Cin_{FA1_i} = Cout_{FA1_{i-1}} \text{ where } i \neq 0$$

$$S_i = S_{FA1_i}$$



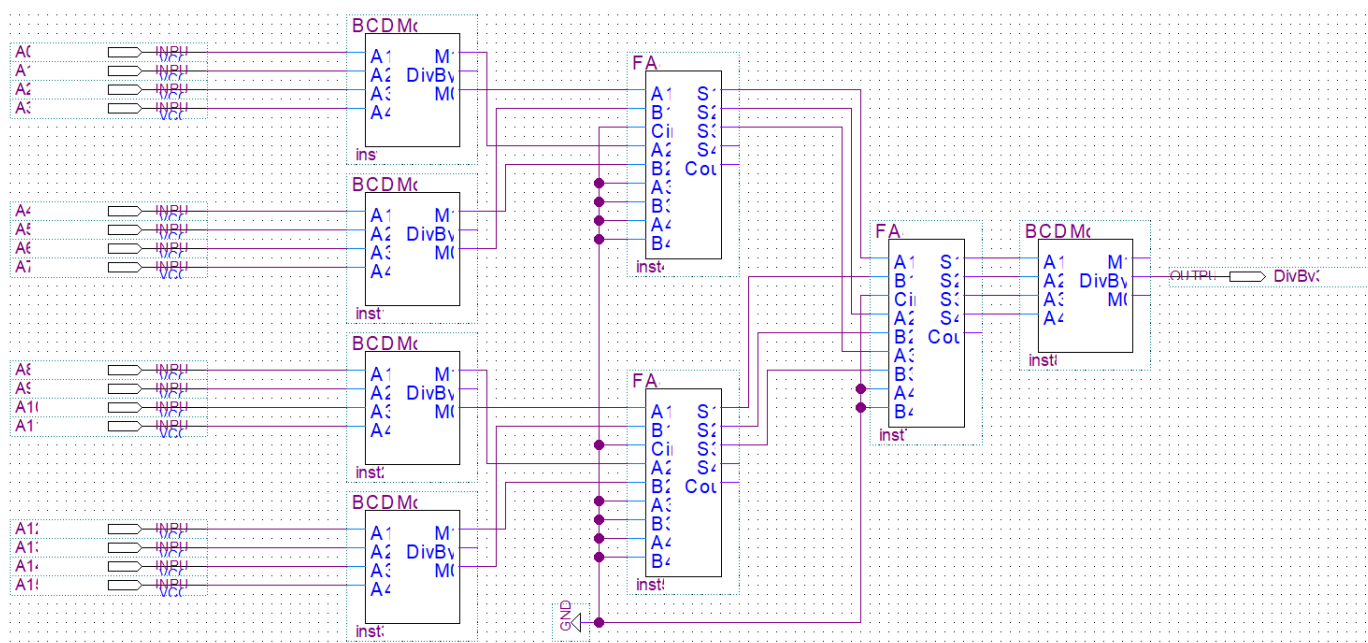
شکل ۷: ماژول FA4



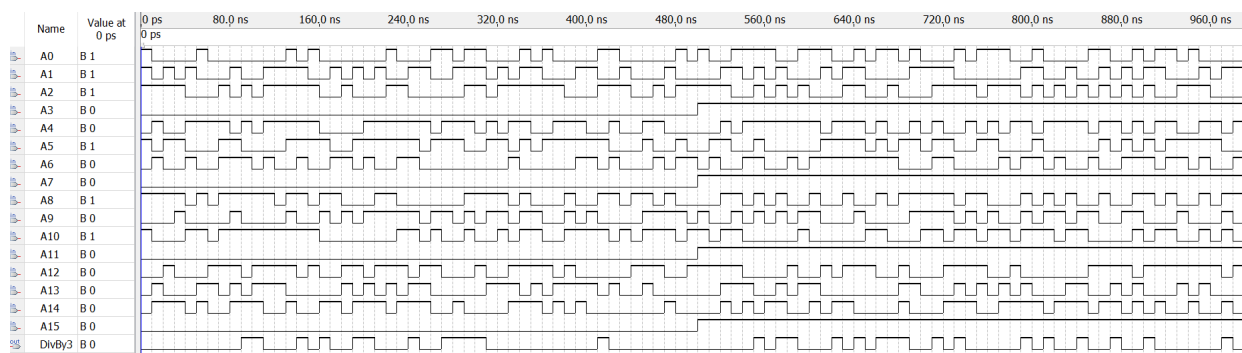
شکل ۸: عملکرد صحیح ماژول FA4

۴.۲ ماژول نهایی Experiment1_DivBy3

مدار نهایی و Top-Level مطابق شکل ۹؟ می باشد که در آن از هیچ گونه گیت استفاده نشده است و صرفاً از اتصال ماژول های طراحی شده، تشکیل شده است.



شکل ۹: ماژول Experiment1_DivBy3



شکل ۱۰: عملکرد صحیح ماژول Experiment1_DivBy3 (توجه شود که مقادیر بصورت تصادفی مقداردهی شدند و ممکن از BCD نباشند. از بررسی آن‌ها در شکل صرف نظر می‌کنیم.)

۳ طراحی مدار تشخیص بخش‌پذیری بر ۱۱ برای عدد ۴-رقمی BCD

۱.۳ هدف مسئله

هدف از این بخش، طراحی یک مدار ترکیبی است که دارای یک خروجی یک‌بیتی باشد. عملکرد این مدار باید به این صورت باشد که خروجی آن تنها زمانی ۱ شود که ورودی BCD چهاررقمی اعمال‌شده، مضربی از عدد ۱۱ باشد. همچنین، محدودیت در نظر گرفته شده برای این طراحی این است که برای پیاده‌سازی مدار، فقط مجاز به استفاده از گیت‌های پایه هستیم.

۲.۳ تئوری مسئله

طبق قواعد ریاضی می‌دانیم عدد $N = \overline{x_1x_2x_3x_4}$ بر عدد ۱۱ بخش‌پذیر است، اگر و تنها اگر مجموع متناوب^۶ ارقام آن بر ۱۱ بخش‌پذیر باشد. به بیان دیگر باید حاصل تفریق جمع ارقام در جایگاه‌های زوج و جمع ارقام در جایگاه‌های فرد آن، مضربی از ۱۱ (که شامل ۰ نیز می‌شود) باشد:

$$N \equiv 0 \pmod{11} \text{ where } N = \overline{x_1x_2x_3x_4} \iff (x_1 + x_3) - (x_2 + x_4) \equiv 0 \pmod{11}$$

۳.۳ پیاده‌سازی

ما به دنبال محاسبه عبارت زیر هستیم:

$$D = (x_1 + x_3) - (x_2 + x_4)$$

تعریف می‌کنیم:

$$S_{\text{even}} = x_1 + x_3$$

$$S_{\text{odd}} = x_2 + x_4$$

$$D = S_{\text{even}} - S_{\text{odd}} = S_{\text{even}} + \overline{S_{\text{odd}}} + 1 \quad (\text{مکمل}^{\vee 2})$$

و با کمی محاسبه نیز، می‌توان نشان داد:

$$-18 \leq D \leq 18$$

و صرفاً کفایت حالت‌های خروجی که در آن $D = 0$ یا $D = -11$ یا $D = 11$ است، بررسی شود. در این پیاده‌سازی برای محاسبه مقادیر S_{even} و S_{odd} نیاز به جمع‌کننده کامل ۴ بیتی داریم. در نهایت حاصل آن ۵ بیتی است و برای مرحله بعد که تفریق است (که ما عملاً جمع می‌کنیم) از جمع‌کننده ۵ بیتی استفاده کنیم، و بخواهیم رقم

نقلی خروجی (Cout) را به عنوان بیت ششم کنار جواب بگذاریم، مدار در محاسبه اعداد منفی دچار خطا می‌شود. (بعنوان مثال اگر $S_{\text{even}} = 0$ و $S_{\text{odd}} = 18$ باشد، حاصل تفریق باید ۱۸- شود. اگر این عملیات را در یک بلوک ۵ بیتی انجام دهیم، سیستم دچار سرریز^۸ شده و مدار به جای عدد ۱۸-، به اشتباه مقدار مثبت ۱۴ را تولید می‌کند.) برای جلوگیری از این خطای محاسباتی، باید پیش از انجام تفریق، اعداد را گسترش علامت^۹ دهیم. با اضافه کردن یک صفر به سمت چپ خروجی‌های ۵ بیتی، آن‌ها را به اعداد ۶ بیتی تبدیل می‌کنیم. سپس این اعداد ۶ بیتی را وارد یک بلوک **جمع‌کننده کامل ۶-بیتی** می‌کنیم. در نهایت خروجی این جمع‌کننده را به ماژول **DivBy11** می‌دهیم تا بررسی کند مقدار D یکی از مقادیر ۰، ۱۱ یا ۱۱- هست یا خیر.

۱.۳.۳ ماژول FA6

در این ماژول از ۶ عدد ماژول **جمع‌کننده کامل ۱-بیتی** استفاده کردیم و آن‌ها را بصورت جمع‌کننده آبشاری^{۱۰} وصل کردیم.

• ورودی‌ها: $A_1, A_2, A_3, A_4, A_5, A_6, B_1, B_2, B_3, B_4, B_5, B_6, Cin$

• خروجی‌ها: $S_1, \overline{S_1}, S_2, \overline{S_2}, S_3, \overline{S_3}, S_4, \overline{S_4}, S_5, \overline{S_5}, S_6, \overline{S_6}, Cout$

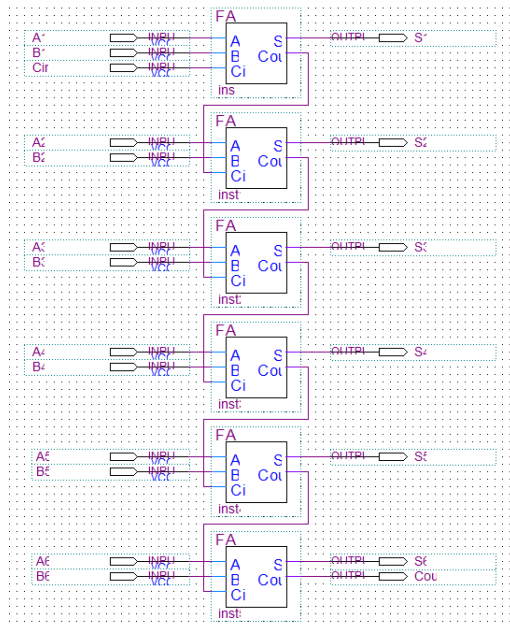
• عملکرد:

$$Cin_{FA1_0} = Cin$$

$$Cout = Cout_{FA1_5}$$

$$Cin_{FA1_i} = Cout_{FA1_{i-1}} \text{ where } i \neq 0$$

$$S_i = S_{FA1_i}$$

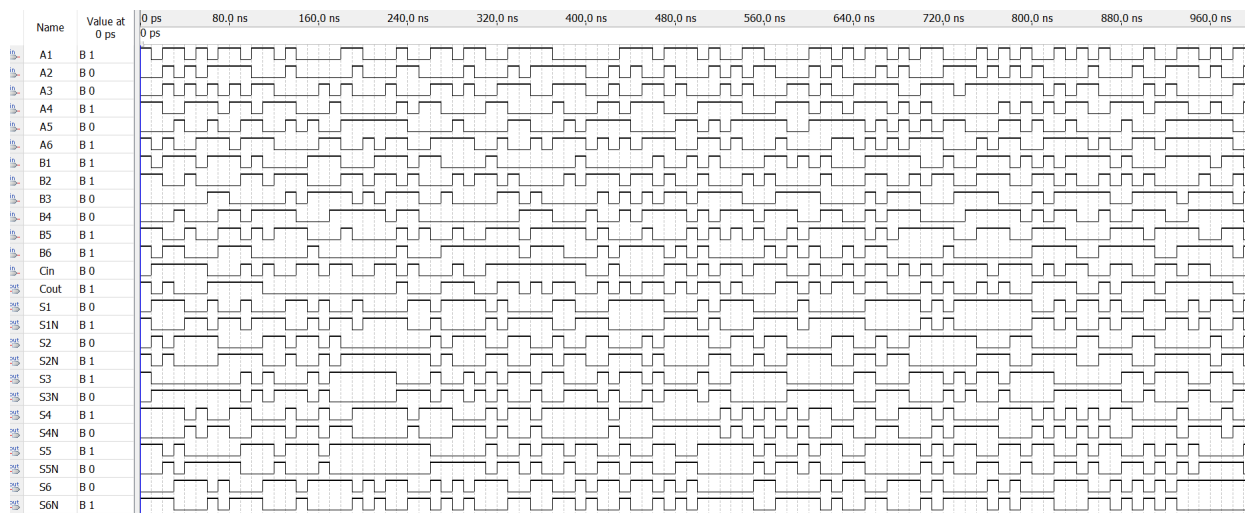


شکل ۱۱: ماژول FA6

^۸Overflow

^۹Sign Extend

^{۱۰}Ripple Carry Adder



شکل ۱۲: عملکرد صحیح ماژول FA6

۲.۳.۳ ماژول DivBy11

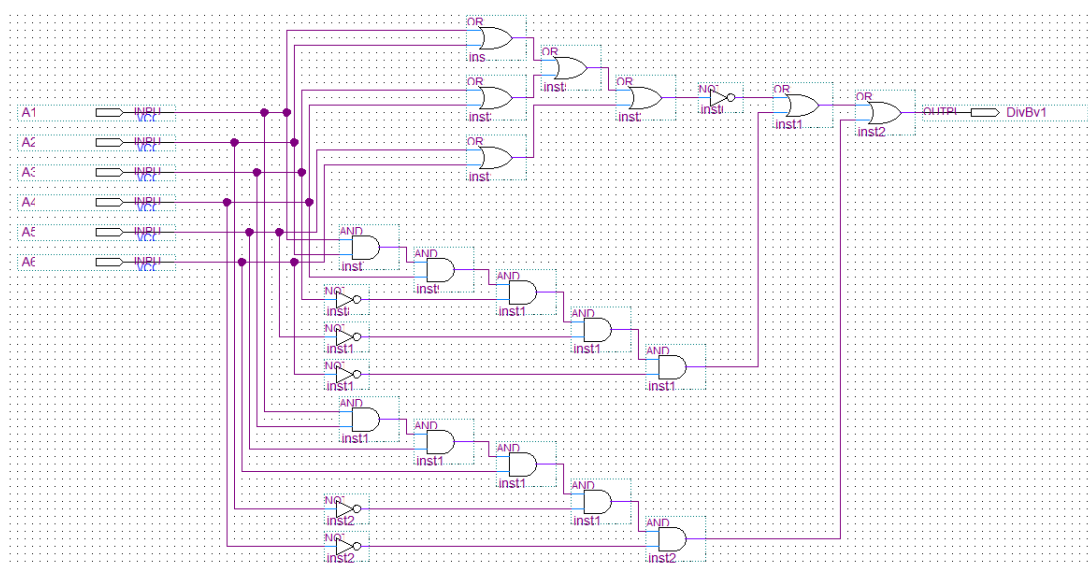
در این ماژول صرفاً بررسی می‌کنیم عدد ورودی بر در طبق محاسبات بالاتر نشان دادیم در بازه [۱۸، ۱۸] قرار گرفته‌است، بر ۱۱ بخش پذیر است یا خیر. برای اینکار کافیت بررسی کنیم عدد ورودی آیا یکی از اعداد ۰، ۱۱ یا ۱۱- است یا خیر.

• ورودی‌ها: $A_1, A_2, A_3, A_4, A_5, A_6$

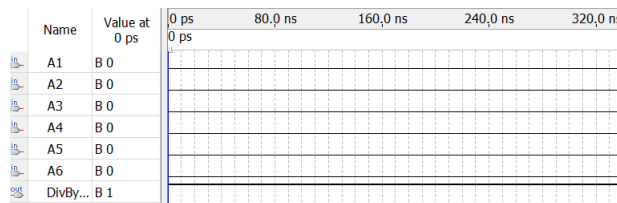
• خروجی‌ها: DivBy11

• عملکرد:

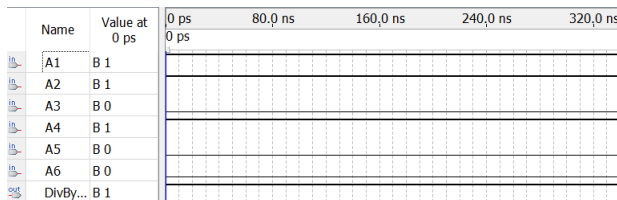
$$\text{DivBy11} = (\overline{A_1 A_2 A_3 A_4 A_5 A_6} == 6b'000000) + (\overline{A_1 A_2 A_3 A_4 A_5 A_6} == 6b'001011) + (\overline{A_1 A_2 A_3 A_4 A_5 A_6} == 6b'110101)$$



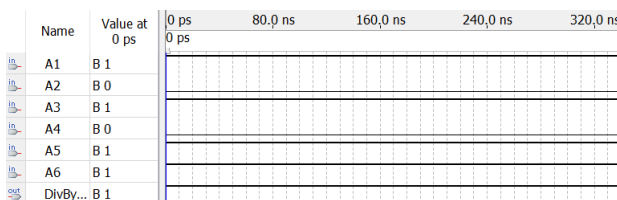
شکل ۱۳: ماژول DivBy11



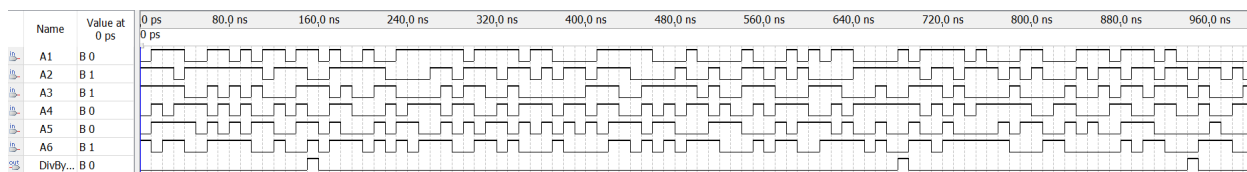
شکل ۱۴: عملکرد صحیح ماژول DivBy11 ورودی 6b'000000



شکل ۱۵: عملکرد صحیح ماژول DivBy11 ورودی 6b'001011



شکل ۱۶: عملکرد صحیح ماژول DivBy11 ورودی 6b'110101



شکل ۱۷: عملکرد صحیح ماژول DivBy11 با ورودی تصادفی

۴.۳ ماژول نهایی Experiment1_DivBy11

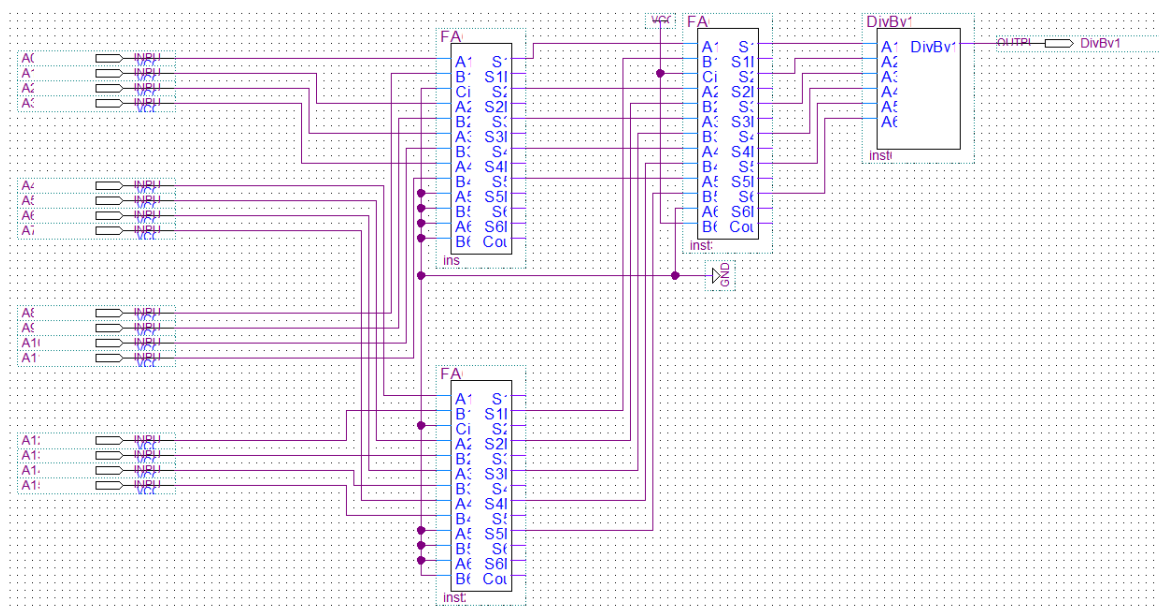
مدار نهایی و Top-Level مطابق شکل ۴۳ می‌باشد که در آن از هیچ‌گونه گیت استفاده نشده است و صرفاً از اتصال ماژول‌های طراحی شده، تشکیل شده است. مطابق توضیحات ابتدا توسط دو **جمع‌کننده کامل ۶-بیتی**، مقادیر S_{even} و S_{odd} محاسبه شدند. سپس مقدار $D = S_{\text{even}} - S_{\text{odd}} = S_{\text{even}} + \overline{S_{\text{odd}}} + 1$ محاسبه شد و حاصل وارد ماژول **DivBy11** شد تا باقی‌مانده نهایی بررسی شود.

نکته مهم در این طراحی این است که چون نمی‌خواستیم پیچیدگی مدار زیاد شود، برای تمام جمع‌ها از همان **جمع‌کننده کامل ۶-بیتی** استفاده کردیم و در مواقعی که نیاز نداشتیم به بیت‌های پرارزش^{۱۱}، آن‌ها را برابر با صفر گذاشتیم. اما نکته بسیار ریز و مهم این است که ما گفتیم قبل از انجام تفریق (یا همان مکمل ۲)، باید گسترش علامت دهیم که چون عدد مدنظر مثبت است عملاً باید یک ۰ اضافه کنیم در ادامه برای انجام عمل تفریق هم باید عدد را قرینه^{۱۲} کنیم و جمع را انجام دهیم. در این طراحی عملیات قرینه‌سازی در خود **جمع‌کننده کامل ۶-بیتی** رخ می‌دهد ولی نکته مهم آن

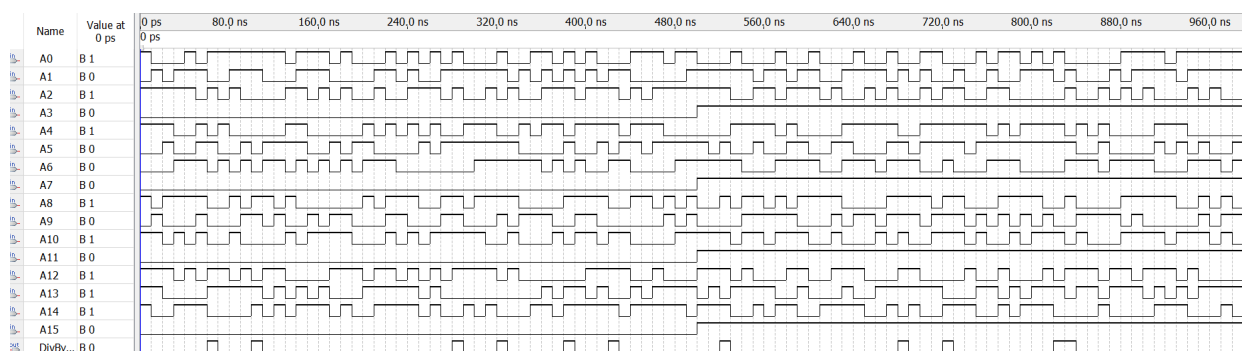
^{۱۱}MSB

^{۱۲}Invert

این است که این اتفاق برای فقط ۵ بیت رخ می‌دهد و سپس گسترش علامت توسط مقدار ۰ انجام می‌گیرد! در صورتی که خود ۰ بی که به جهت گسترش علامت اضافه شده بود نیز باید قرینه می‌شد! بنابراین ما به جای اینکه به ورودی ششم **جمع‌کننده کامل ۶-بیتی** سوم (که هدفش محاسبه $D = S_{\text{even}} + \overline{S_{\text{odd}}} + 1$ بود) بجای مقدار ۰ (یا همان gnd) مقدار ۱ (یا همان vcc) را وصل کردیم.



شکل ۱۸: ماژول Experiment11_DivBy11



شکل ۱۹: عملکرد صحیح ماژول Experiment11_DivBy3 (توجه شود که مقادیر بصورت تصادفی مقداردهی شدند و ممکن از BCD نباشند. از بررسی آن‌ها در شکل صرف نظر می‌کنیم.)